

# Build Your Own AI Coding Agent with GitHub Copilot

Juezhao Yu

jz\_yu@nus.edu.sg

- We have learned how to debug code manually and how to work with GitHub Copilot.
- Now, let's see how coding agents automate the coding and debugging loop — by building a minimal coding agent ourselves.

- Build a minimal AI coding agent using GitHub Copilot.
- Learn how modern coding agents understand repositories, plan changes, edit code, and verify their work.
- Develop practical AI software engineering skills for collaborating effectively with coding agents.
- AI-native thinking: **USE AI TO UNDERSTAND AND BUILD AI**

- Three ways you can run an LLM
- Run it locally on your laptop
- Run it on the cloud by calling an API from a service provider
- Run it indirectly by using LLM-backed product (e.g. GitHub Copilot, ChatGPT, ...)

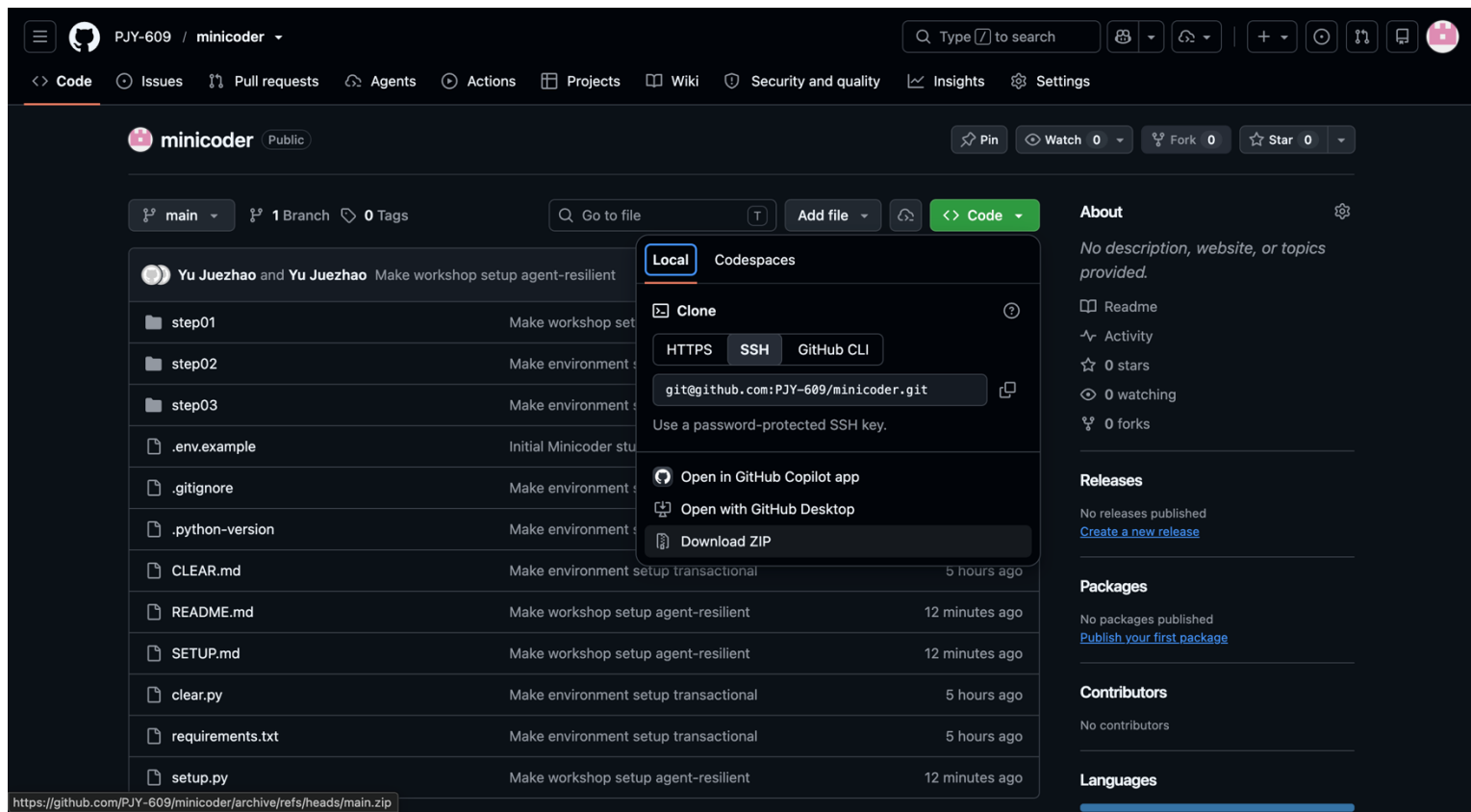
# Outline

- STEP01: running a local LLM on your laptop
- STEP02: calling a cloud LLM through API
- STEP03: build a mini coding agent with a cloud LLM

# Prerequisite

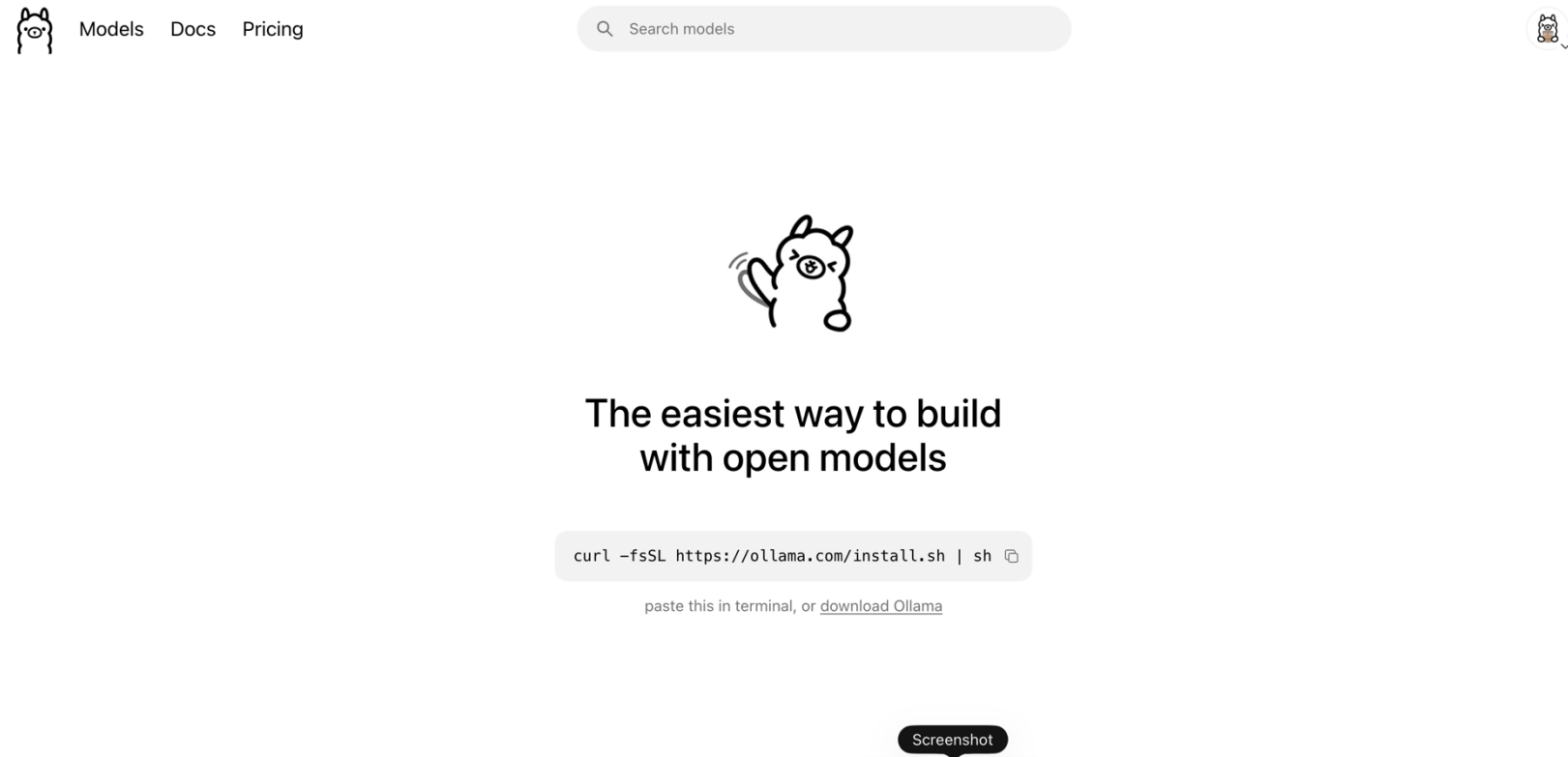
- GitHub Copilot required for all three steps.
  - Codex or Claude Code also works
  - It's OK if you still do not have access to it yet. We will share the materials and recording with you after the session. You can just watch me demo and try on your own afterwards.
- VS Code and a working Python environment (Anaconda)
- STEP01: running a local LLM on your laptop
  - Need to download ollama, but you can skip this step. Local LLMs are pretty weak. They cannot handle complex task. I will show you.
- STEP02: calling a cloud LLM through API
  - Need to top up in OpenRouter. The minimum top up rate is 5 USD.
- STEP03: build a mini coding agent with a cloud LLM
  - Same as STEP02.

- Check out
- <https://github.com/PJY-609/minicoder>



# STEP01: running a local LLM on your laptop

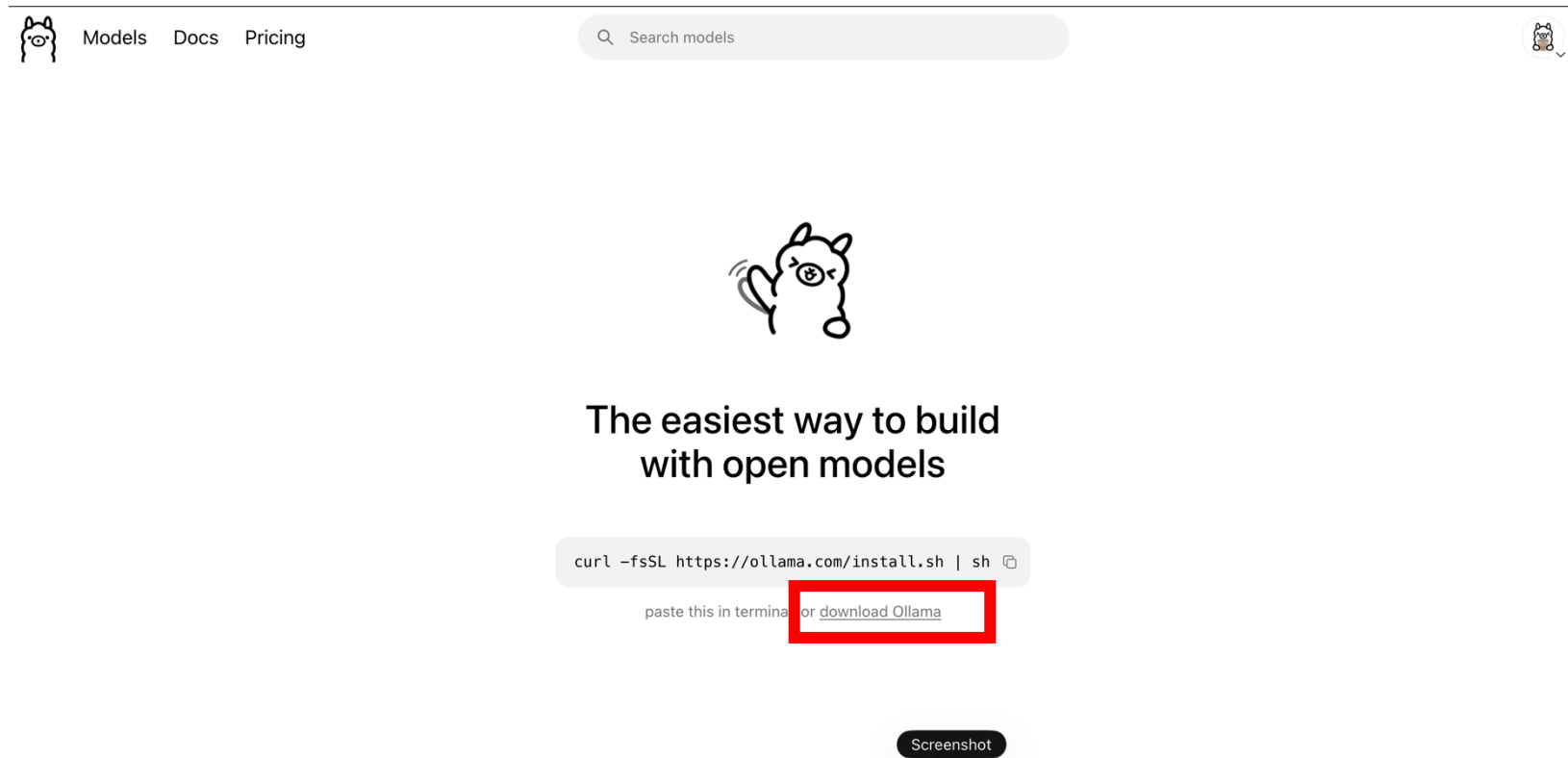
- Download Ollama
- Search “Ollama”



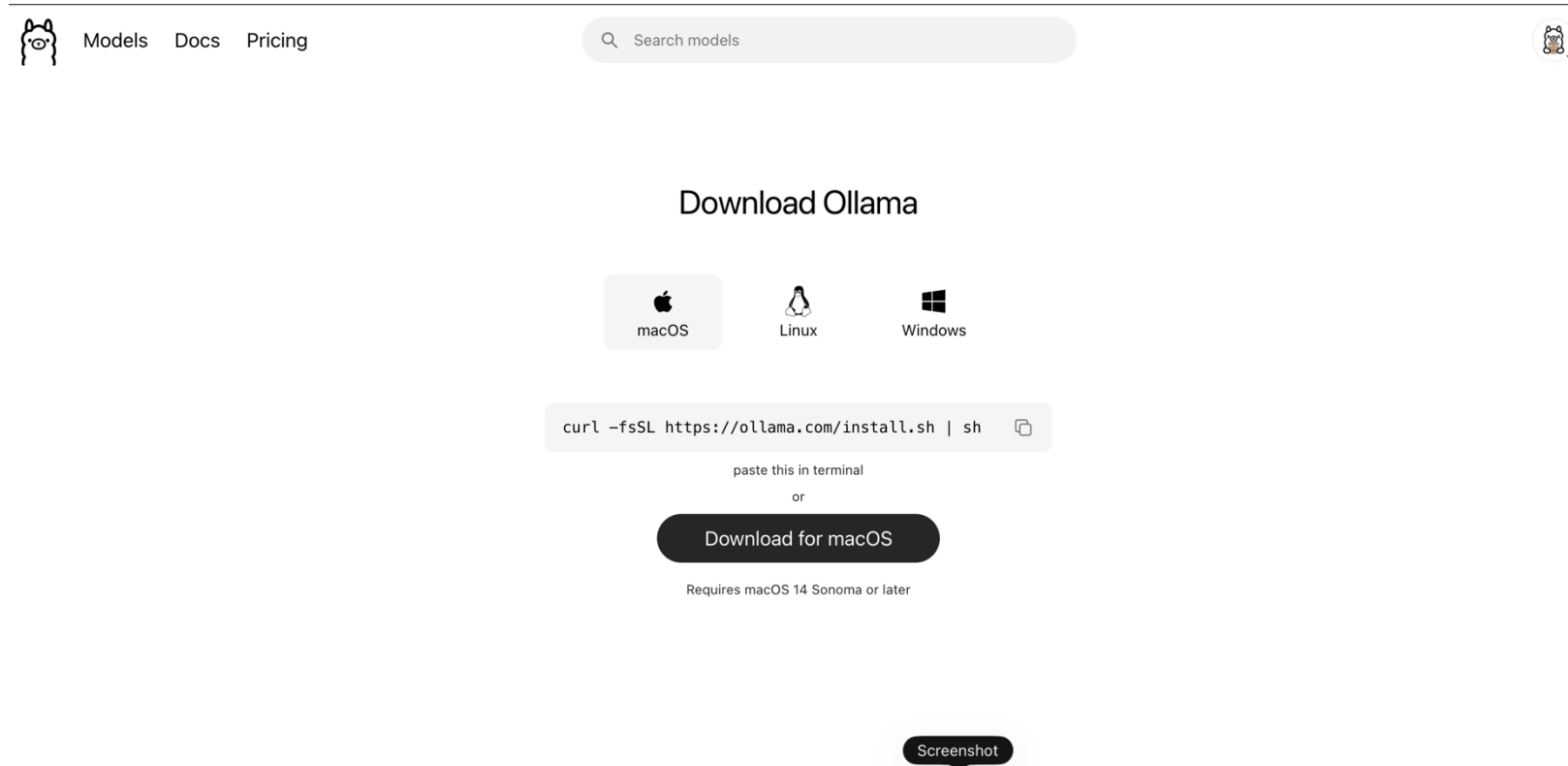


# STEP01: running a local LLM on your laptop

- If you don't know how to use the terminal,



# STEP01: running a local LLM on your laptop



# STEP01: running a local LLM on your laptop

- It takes a while to download and install.
- If it doesn't work on your laptop, just watch the demo.

# STEP01: running a local LLM on your laptop

- Ask GitHub Copilot to act on SETUP.md
- It will help examine whether your ollama is properly installed or not.
- If ollama is properly installed, then it will further help you install the local LLM we need with ollama.
- It will also help setup the python environments for us.
- It takes a while.
- Please just click “allow” for the actions that GitHub Copilot is going to take.
- If it asks you to choose a Python environment (we assume you have set up earlier), choose the one with the latest Python version (if you have many)

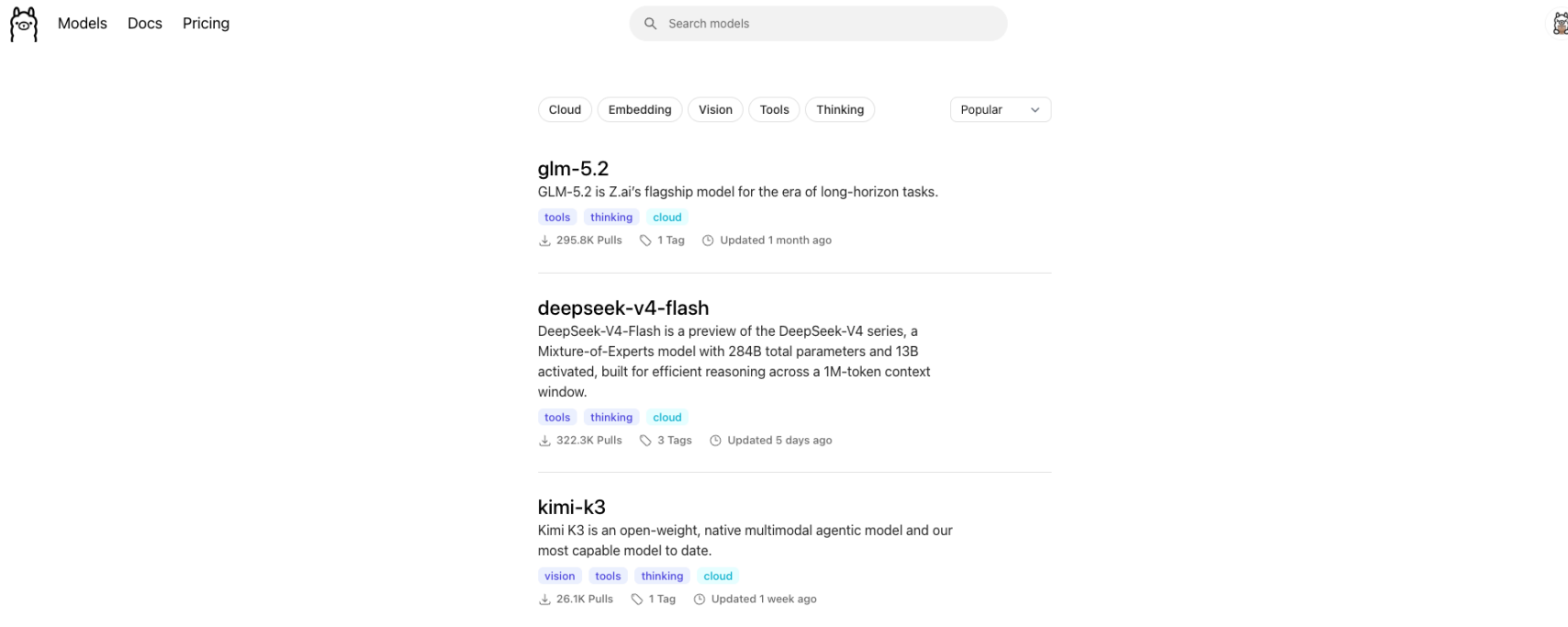
# STEP01: running a local LLM on your laptop

- We will use Qwen3 0.6B to run on local laptop.

| Practical requirements                                                                                                                                                            |                          |                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|------------------------------|
| Configuration                                                                                                                                                                     | Minimum                  | Recommended                  |
| System RAM                                                                                                                                                                        | 4 GB                     | 8 GB or more                 |
| Free storage                                                                                                                                                                      | 1–2 GB                   | 3–5 GB                       |
| Dedicated GPU                                                                                                                                                                     | Not required             | Optional                     |
| GPU VRAM                                                                                                                                                                          | Not required             | 2 GB or more                 |
| CPU                                                                                                                                                                               | Modern 64-bit dual-core  | Recent 4-core or better      |
| Operating system                                                                                                                                                                  | Windows, macOS, or Linux | Any recent version           |
| For comparison, Ollama's quantized <code>qwen3:0.6b</code> download is approximately 523 MB.  |                          |                              |
| Estimated memory usage                                                                                                                                                            |                          |                              |
| The actual memory depends on the model format and context length.                                                                                                                 |                          |                              |
| Model format                                                                                                                                                                      | Weight memory            | Typical total RAM during use |
| FP32                                                                                                                                                                              | About 2.4 GB             | 3.5–5 GB                     |
| FP16/BF16                                                                                                                                                                         | About 1.2 GB             | 2–4 GB                       |
| INT8                                                                                                                                                                              | About 0.6–0.8 GB         | 1.5–3 GB                     |
| INT4, such as Ollama/GGUF                                                                                                                                                         | About 0.5–0.7 GB         | 1–2.5 GB                     |

- An 8 GB laptop should run it easily.
- A 16 GB laptop has ample headroom.
- A 4 GB laptop may run the quantized version, but the operating system could create memory pressure.
- An integrated Intel, AMD, or Apple GPU is sufficient; CPU-only inference also works.

# STEP01: running a local LLM on your laptop



# STEP01: running a local LLM on your laptop

| Current frontier-model sizes |              |                  |                   |                     |
|------------------------------|--------------|------------------|-------------------|---------------------|
| Model                        | Architecture | Total parameters | Active per token  | Publicly disclosed? |
| GPT-5.6 Sol                  | Undisclosed  | Unknown          | Unknown           | No                  |
| Claude Fable 5               | Undisclosed  | Unknown          | Unknown           | No                  |
| Claude Mythos 5              | Undisclosed  | Unknown          | Unknown           | No                  |
| Kimi K3                      | MoE          | 2.8T             | 104B              | Yes                 |
| DeepSeek V4 Pro              | MoE          | 1.6T             | 49B               | Yes                 |
| DeepSeek V4 Flash            | MoE          | 284B             | 13B               | Yes                 |
| Qwen3.8-Max                  | MoE          | >1000x 2.4T      | Approximately 95B | Yes/reported        |
| Qwen3-0.6B                   | Dense        | 0.6B             | 0.6B              | Yes                 |

# STEP01: running a local LLM on your laptop

- Task: we would like to ask Qwen3 0.6B the following question picked from Winograd Schema Challenge

```
WSC_QUESTION = (  
    "Winograd Schema Challenge question "  
    "(Hugging Face dataset: winograd_wsc, config wsc273):\n\n"  
    "Sentence: The trophy doesn't fit into the brown suitcase "  
    "because it is too big.\n\n"  
    "Question: What is too big \u2014 the trophy or the suitcase?\n\n"  
    "Give your answer and a one-sentence justification. You do "  
    "not need to verify whether your answer matches the "  
    "dataset's official label."  
)
```

What should be the correct answer?



# STEP01: running a local LLM on your laptop

- Task: we would like to ask Qwen3 0.6B the following question picked from Winograd Schema Challenge
- Winograd Schema Challenge proposes a dataset to ask computer (AI) to answer a certain type of questions regarding the semantic understanding of natural language.
- AI didn't well in answering this kind of questions until we have LLMs.

```
WSC_QUESTION = (  
    "Winograd Schema Challenge question "  
    "(Hugging Face dataset: winograd_wsc, config wsc273):\n\n"  
    "Sentence: The trophy doesn't fit into the brown suitcase "  
    "because it is too big.\n\n"  
    "Question: What is too big \u2014 the trophy or the suitcase?\n\n"  
    "Give your answer and a one-sentence justification. You do "  
    "not need to verify whether your answer matches the "  
    "dataset's official label."  
)
```

# STEP01: running a local LLM on your laptop

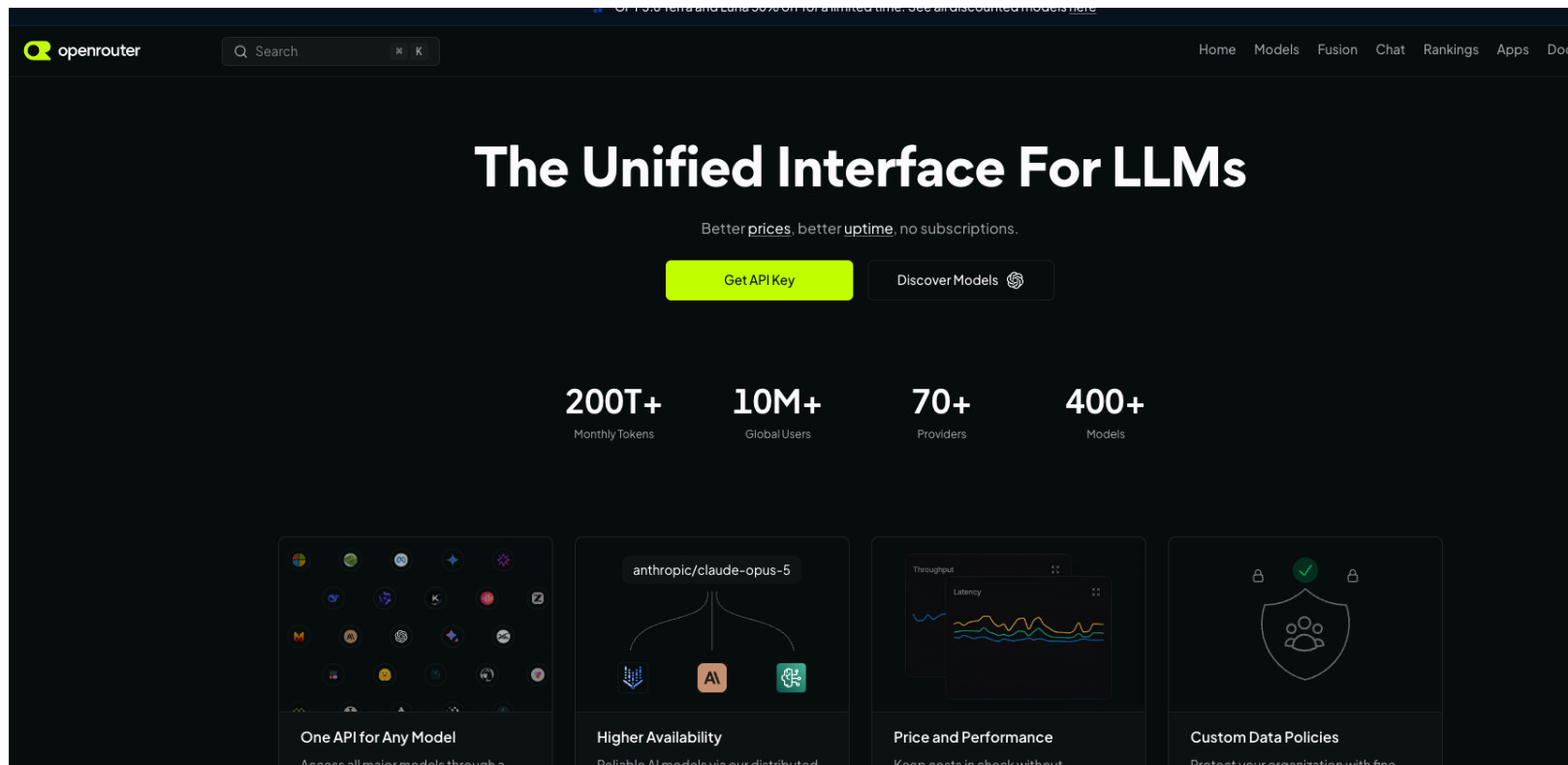
- Basic features of GitHub Copilot
- Ask: understand the code by clarifying with GitHub Copilot
- Plan: plan before complex tasks (you can refine the plan interactively until you deem appropriate to execute)
- Agent: “autopilot”
- And more....

# STEP01: running a local LLM on your laptop

- What is the output of qwen3:0.6b?
- Can it produce the correct answer?
- We change to a larger model.

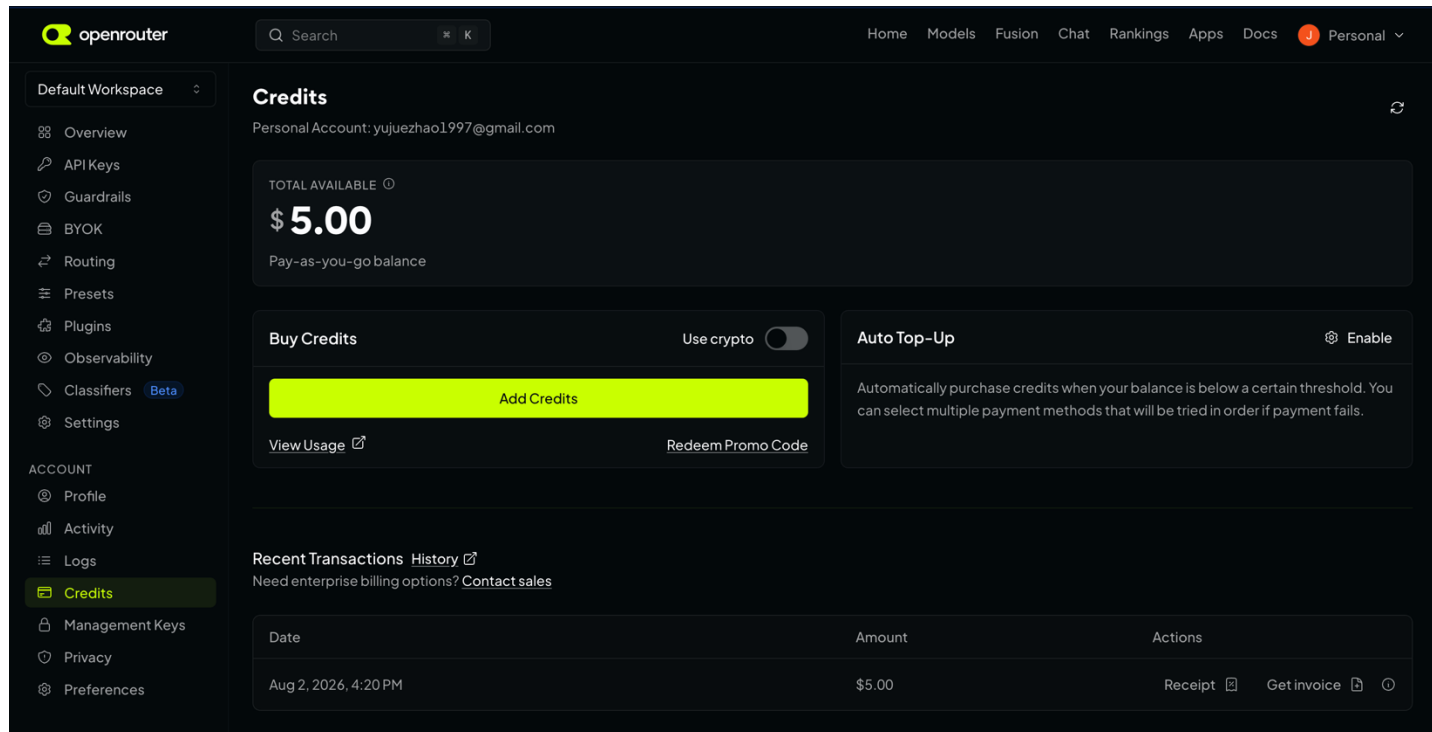
# STEP02: calling a cloud LLM through API

- If you want to follow through, you will need to sign up on OpenRouter



# STEP02: calling a cloud LLM through API

- We will need to top up some \$ if we want to call the frontier models. The min top up rate is 5 USD, which is more than enough for our mini project.



The screenshot shows the OpenRouter 'Credits' page for a personal account. The page displays a total available balance of \$5.00, which is a pay-as-you-go balance. There are two main sections: 'Buy Credits' and 'Auto Top-Up'. The 'Buy Credits' section has a toggle for 'Use crypto' and a prominent yellow 'Add Credits' button. Below it are links for 'View Usage' and 'Redeem Promo Code'. The 'Auto Top-Up' section has an 'Enable' toggle and a description of the feature. At the bottom, there is a 'Recent Transactions' table with one entry for a \$5.00 transaction on August 2, 2026.

**openrouter** Search K Home Models Fusion Chat Rankings Apps Docs Personal

Default Workspace

**Credits** Personal Account: yujuezhao1997@gmail.com

TOTAL AVAILABLE ⓘ

**\$5.00**

Pay-as-you-go balance

**Buy Credits** Use crypto ☐

**Add Credits**

[View Usage](#) [Redeem Promo Code](#)

**Auto Top-Up** Enable

Automatically purchase credits when your balance is below a certain threshold. You can select multiple payment methods that will be tried in order if payment fails.

**ACCOUNT**

- Profile
- Activity
- Logs
- Credits**
- Management Keys
- Privacy
- Preferences

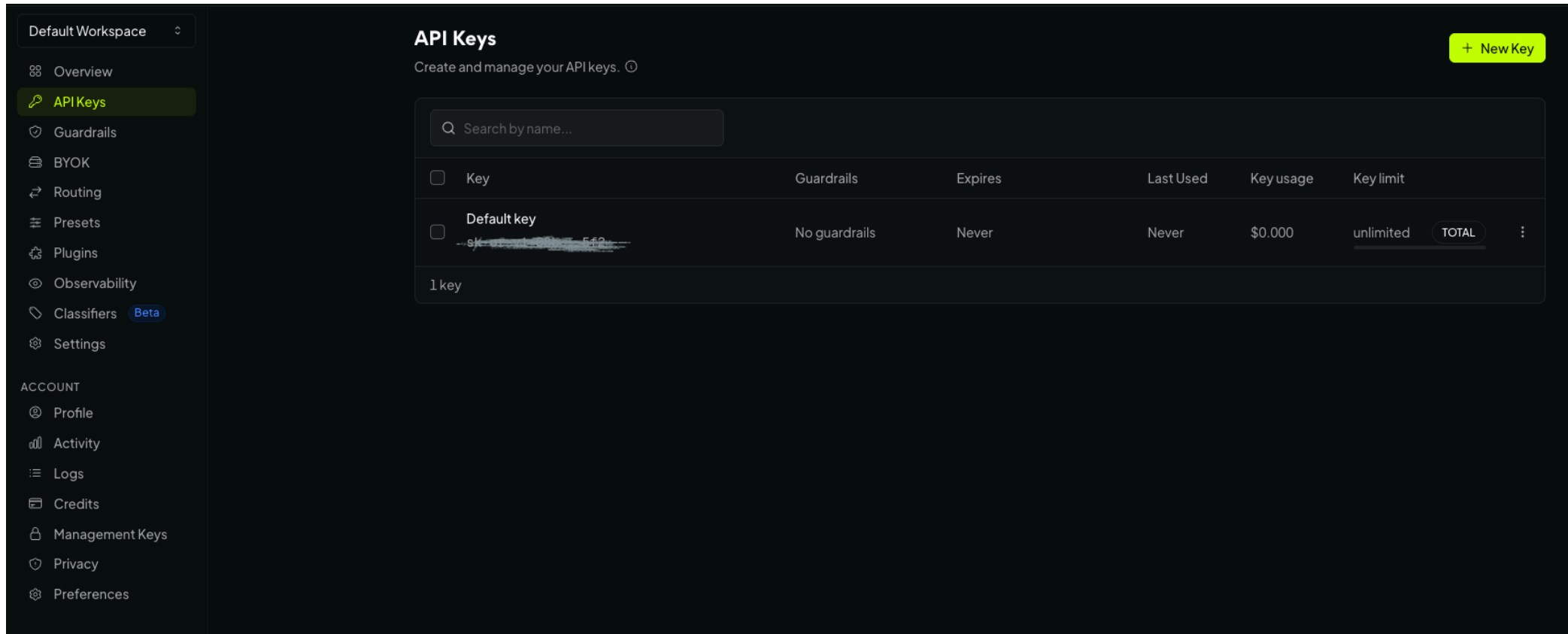
**Recent Transactions** [History](#)

Need enterprise billing options? [Contact sales](#)

| Date                 | Amount | Actions                                                            |
|----------------------|--------|--------------------------------------------------------------------|
| Aug 2, 2026, 4:20 PM | \$5.00 | <a href="#">Receipt</a> <a href="#">Get invoice</a> <span>ⓘ</span> |

# STEP02: calling a cloud LLM through API

- Next, we will need to create an API token.



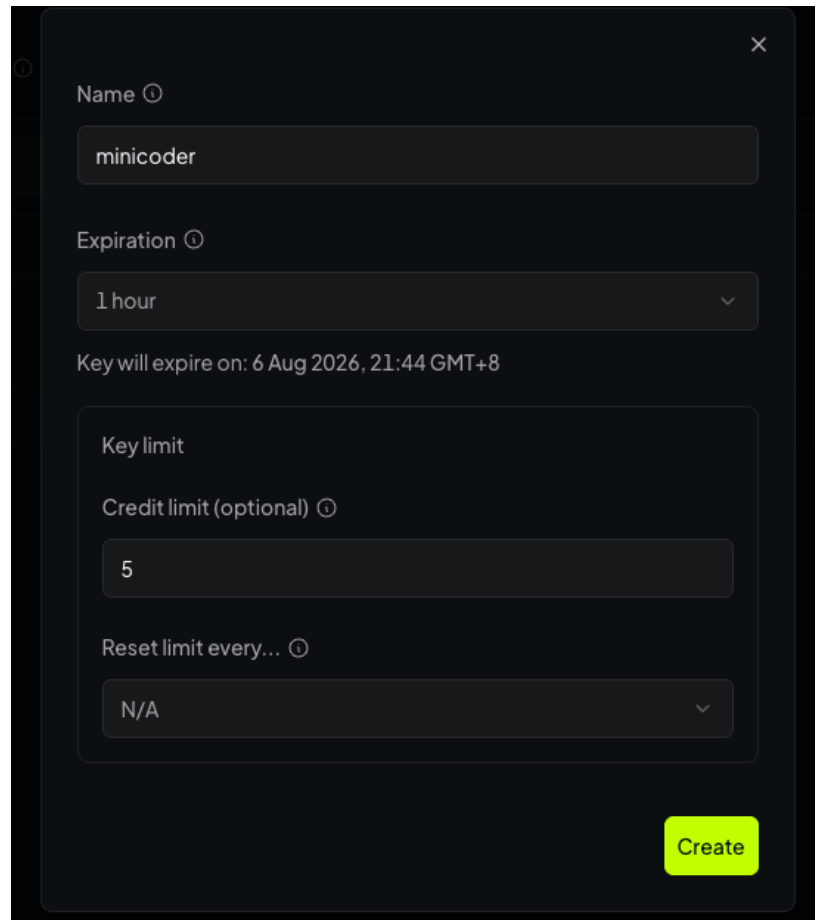
The screenshot displays the 'API Keys' management page in a dark-themed application. On the left, a sidebar contains a navigation menu with options: Overview, API Keys (highlighted), Guardrails, BYOK, Routing, Presets, Plugins, Observability, Classifiers (marked as Beta), and Settings. Below this, an 'ACCOUNT' section lists Profile, Activity, Logs, Credits, Management Keys, Privacy, and Preferences. The main content area is titled 'API Keys' with a subtitle 'Create and manage your API keys.' and a '+ New Key' button. A search bar labeled 'Search by name...' is positioned above a table. The table has columns for selection, Key, Guardrails, Expires, Last Used, Key usage, and Key limit. It contains one entry, 'Default key', which is associated with a redacted API key, 'No guardrails', and 'Never' expiration. The 'Key usage' is '\$0.000' and the 'Key limit' is 'unlimited'. A 'TOTAL' button and a three-dot menu are located at the end of the row. A summary bar at the bottom of the table indicates '1 key'.

|                          | Key                              | Guardrails    | Expires | Last Used | Key usage | Key limit |
|--------------------------|----------------------------------|---------------|---------|-----------|-----------|-----------|
| <input type="checkbox"/> | Default key<br><div>sk-...</div> | No guardrails | Never   | Never     | \$0.000   | unlimited |

1 key

# STEP02: calling a cloud LLM through API

- Set up the token as the following

A screenshot of a dark-themed web interface for creating a new API token. The interface includes a close button (X) in the top right corner. It has three main sections: 'Name' with a text input containing 'minicoder', 'Expiration' with a dropdown menu set to '1 hour' and a note 'Key will expire on: 6 Aug 2026, 21:44 GMT+8', and a 'Key limit' section containing 'Credit limit (optional)' with a text input set to '5' and 'Reset limit every...' with a dropdown menu set to 'N/A'. A yellow 'Create' button is located at the bottom right.

Name ⓘ

minicoder

Expiration ⓘ

1 hour

Key will expire on: 6 Aug 2026, 21:44 GMT+8

Key limit

Credit limit (optional) ⓘ

5

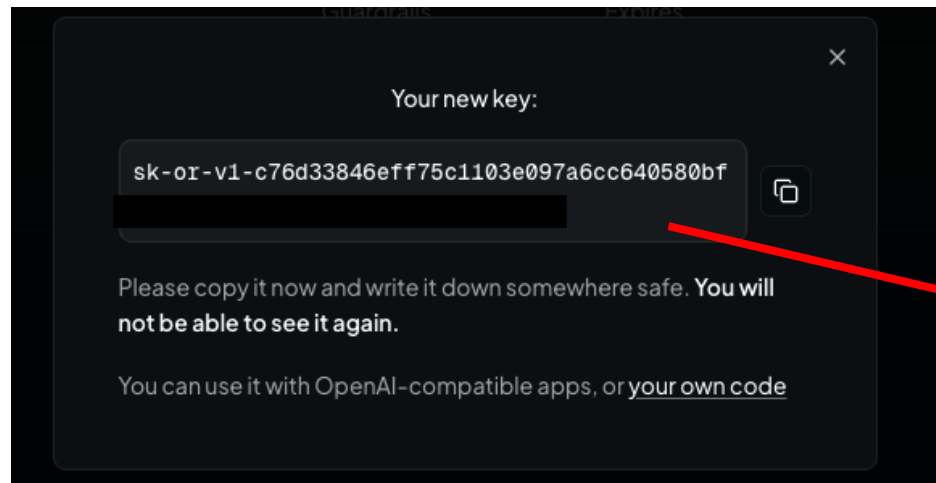
Reset limit every... ⓘ

N/A

Create

# STEP02: calling a cloud LLM through API

- You will see your token. Please copy-paste it into the .env file.
- The .env file should be generated in the project folder after you successfully run GitHub Copilot on SETUP.md.
- If you cannot find .env, it means either you didn't run or the run wasn't successful.



```
1  OLLAMA_HOST=http://localhost:11434
2  OLLAMA_API_KEY=
3  OLLAMA_MODEL=qwen3:0.6b
4
5  # Cloud backend used by milestone 03's multi-round session
6  # Get an API key at https://openrouter.ai/keys, then paste
7  # OPENROUTER_MODEL must be the exact model slug shown on h
8  LLM_BACKEND=ollama
9  OPENROUTER_API_KEY=
10 OPENROUTER_MODEL=
11
12
```



# STEP02: calling a cloud LLM through API

- We will use DeepSeek V4 Flash
- Please put “~deepseek/deepseek-v4-flash-latest” in .env file.

```
1 OLLAMA_HOST=http://localhost:11434
2 OLLAMA_API_KEY=
3 OLLAMA_MODEL=qwen3:0.6b
4
5 # Cloud backend used by milestone 03's multi-round session
6 # Get an API key at https://openrouter.ai/keys, then paste
7 # OPENROUTER_MODEL must be the exact model slug shown on ht
8 LLM_BACKEND=ollama
9 OPENROUTER_API_KEY= sk-or-v1-c76d33846eff75c1103e097a6cc640580bf
10 OPENROUTER_MODEL=~deepseek/deepseek-v4-flash-latest
11
12 |
```

# STEP02: calling a cloud LLM through API

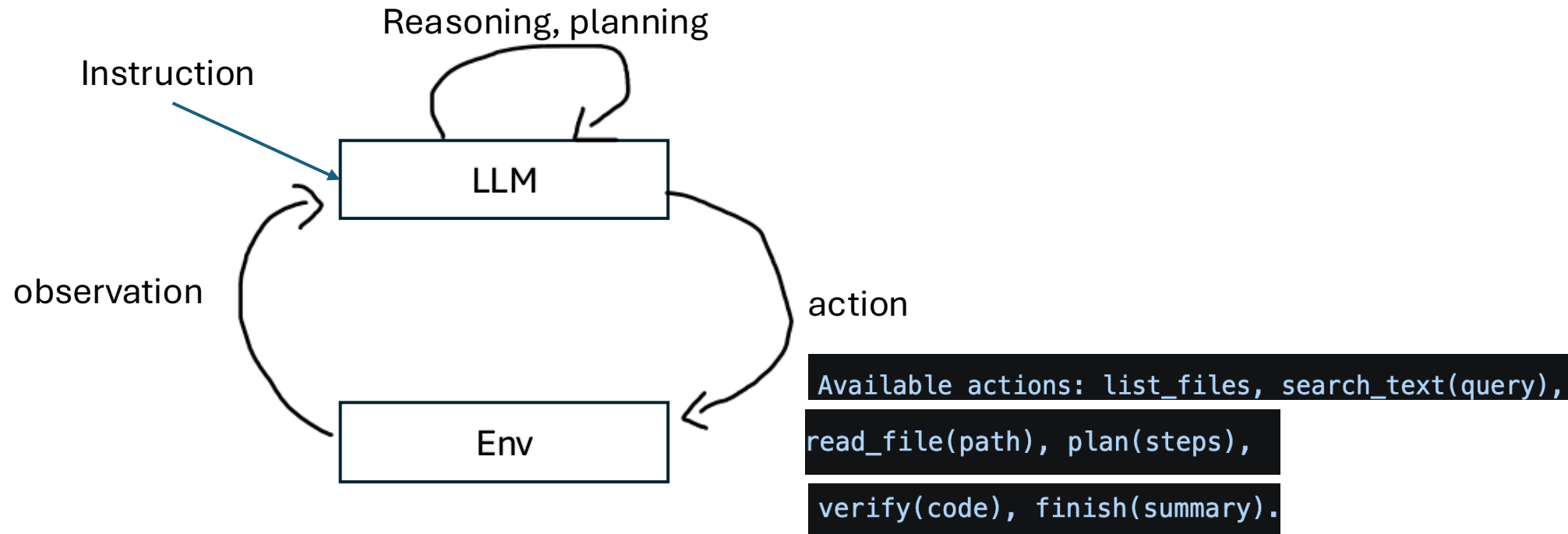
- Observe the output of DeepSeek V4 Flash (284B total parameters)
- Can it answer the question correctly?
- Model scale matters!

For modern, well-trained dense language models:

| Model scale             | Typical qualitative capability                                                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Below 1B                | Language fluency, memorized knowledge and simple instruction following; reasoning is fragile                                                                    |
| 1–3B                    | Basic generalization and elementary reasoning begin to become practically useful                                                                                |
| 7–14B                   | First range that often feels like a broadly capable general-purpose assistant                                                                                   |
| 30–70B                  | More reliable reasoning, coding, knowledge integration and instruction following                                                                                |
| 100B+ active parameters | Historically associated with strong few-shot and chain-of-thought abilities, although newer training methods can obtain similar abilities at much smaller sizes |
| Frontier MoE models     | Tens to roughly 100B active parameters, despite hundreds of billions or trillions of stored parameters                                                          |

# STEP03: build a mini coding agent with a cloud LLM

- A coding agent backed by LLM works in the following way



## STEP03: build a mini coding agent with a cloud LLM

- Let's take a quick look at how the actions are defined as python code.
- LLM is supposed to reply in JSON format (key-value pair), which is a common practice for agentic LLMs. It's easier to manage the response.

## STEP03: build a mini coding agent with a cloud LLM

- Then we look at the coding question a mini coding agent is trying to solve.

# STEP03: build a mini coding agent with a cloud LLM

- After the agent is done, we take a look at the interaction between the LLM and the environment (in this case, our laptop)
- How many steps did the agent take?
- What is the purpose of each step?
- How did we make sure the final output of LLM is reliable?

# STEP03: build a mini coding agent with a cloud LLM

- We have a test suite to support the action “verify”!
- If the LLM fails in the first attempt when calling verify, it will receive the error message and reason by itself and plan new actions and try again.
- It will iterate over this loop until it passes the “verify” step.
- What does above mentioned process tell us?

# STEP03: build a mini coding agent with a cloud LLM

- **We Need to Redefine Debugging in the Era of Agentic AI**
- **Traditional Debugging:** Read the error message. -> Diagnose the root cause. -> Modify the code. -> Run the program again. -> Repeat until it works.
- **Today, coding agents can automate much of this entire loop.**



# STEP03: build a mini coding agent with a cloud LLM

- **So What Does "Debugging" Mean Now?**
- The engineer's role shifts from **fixing code** to **providing verifiable evidence** that the code is correct.
- Instead of spending most of our time writing fixes, we increasingly spend our time defining how the solution should be verified, for example by:
  - Writing comprehensive test suites.
  - Specifying acceptance criteria.
  - Designing edge cases and failure scenarios.
- **In the age of coding agents, debugging is no longer just fixing bugs—it is designing the verification process that allows AI to find and fix bugs correctly.**

# STEP03: build a mini coding agent with a cloud LLM

- What if AI can provide its own verifiable mechanism?
- ...
- There are many more interesting questions waiting for you to explore.

# Wrapping up

- We learn how to
- Use GitHub Copilot to approach a tiny project
- Run a local LLM on laptop
- Run a cloud LLM via API
- Build a mini coding agent backed by cloud LLM.